

AI Coding Agent — Super Orchestrator Guide

Audience: all four bootcamp tracks (PM / BA / SA / Dev)

Companion to: `PM-AI-Bootcamp-Program-v1.0.md`, the four workbooks, and `PM-AI-Bootcamp-OperatingModel-v1.0.md`

Core message: do not treat an AI coding agent as a question-answer chatbot. Use it as an **orchestrator** that can carry a whole step: read context, plan, call tools, run commands, self-check, and log. **You** remain the conductor, the judge, and the accountable owner.

Applies to: Claude Code, Codex, Cursor, or any equivalent AI coding agent — the orchestration principles are **tool-agnostic**; only the feature toggles differ between tools.

Terminology: for the full unified glossary with industry-standard mapping, see `docs/GLOSSARY.md`.

Date: 2026-08-24 · **Version:** v2.0

o. One-minute summary — what "orchestrator" means

A normal chatbot loop looks like this:

- you ask one question,
- it returns one paragraph,
- you do the rest outside the tool.

An AI coding agent as orchestrator is different:

- you give it a **multi-step goal**,
- it breaks the work down,
- reads files,
- writes files,
- runs commands,
- calls external tools,
- can spawn subagents in parallel,
- self-checks the result,
- and reports back.

You do **not** micromanage each command. You set the goal, set the rails, approve at gates, and catch wrong reasoning.

Analogy: you are the conductor, the AI coding agent is the orchestra plus a section coordinator. You do not play every instrument yourself; you direct timing, intent, and correction.

Tool-agnostic: this guide uses Claude Code as the running example, but the principles (seven-beat loop, Leash A/A+, fail-closed gate) apply to **any AI coding agent** — Codex, Cursor, Windsurf, etc. What changes between tools is how you toggle features (plan mode, subagent, hook); what stays the same is **your orchestration mindset and governance**.

1. Mindset shift: from "ask-answer" to "delegate-orchestrate"

Old chatbot behavior	AI coding agent as orchestrator
Ask one question, copy the answer elsewhere	Give one goal and let it work directly on the project
You connect all the steps manually	It plans and executes a multi-step flow
The result lives only in chat	The result becomes a real artifact : a file, code, tests, a dashboard
You either trust it or not	You add explicit gates and verify before use
One thing at a time	It can use parallel subagents on larger work

Rule of thumb:

"Do not ask it to write a paragraph for me. Ask it to finish the whole step, then I inspect it."

2. Standard orchestrator loop — the seven-beat cycle

Use this loop for every type of work: requirement writing, architecture, coding, testing, reporting, and capstone execution.

1. **Context** — load the playground: project files, `CLAUDE.md`, current constraints, target artifacts.
2. **Plan** — ask the AI coding agent to present a step-by-step plan and wait for your approval.
3. **Delegate** — define the boundaries: what it may change, what it must not touch, which tasks are A vs A+.
4. **Execute** — let it run the step end-to-end.

- 5. **Gate / Verify** — tests, lint, traceability, security checks, human review. *If you cannot verify it, do not use it.*
- 6. **Log** — write Dev Book notes and telemetry immediately.
- 7. **Iterate** — loop to the next step.

This is the working muscle memory of the bootcamp:
Context -> Plan -> Delegate -> Execute -> Gate -> Log -> Iterate

3. Eight controls to learn early

Control	What it is	Why it matters
Context file (CLAUDE.md / .codex / equivalent)	Project memory file	Gives the agent stable context and rules
Plan mode	"Plan first, wait for approval" mode	Makes large or risky work cheaper to correct early
Subagents / parallel work	Multiple helpers working at once	Useful for reviews, multi-file analysis, parallel tasks
MCP tools	External tools, browsers, docs, APIs	Lets the AI coding agent interact with real systems
Skills / slash commands	Pre-packaged workflows	Good for standardized tasks
Hooks	Automatic commands after actions	Useful for built-in rails like tests or lint
Memory	Cross-session preferences and decisions	Helps the agent remember how your project works
Permission mode	How much it can do without asking	This is where leash A / A+ becomes real

Non-coder note: you do not need to memorize syntax. If you are stuck, ask the AI coding agent directly to explain the error or tell you how to turn on the right operating mode.

4. How the AI coding agent supports each workbook type

Track / step	AI coding agent does	Human still decides
BA - requirements / user stories	Drafts requirements, finds contradictions, writes user stories and AC	Which questions are real, which assumptions are unsafe, where to hard-stop

Track / step	AI coding agent does	Human still decides
BA - prototype	Builds a clickable prototype or simple flow	Whether it actually meets acceptance criteria
SA - architecture / ADR	Generates options, diagrams, contracts, data models	Which trade-off to accept and why
SA - vertical slice	Builds a technical skeleton from the approved architecture	Whether boundaries and NFRs are truly respected
Dev - code + tests	Writes code, generates tests, helps debug	PR-level review, fake-test detection, correctness judgment
Dev - harness / agent	Builds a small agent or harness for repeated work	Where gates belong and when autonomy is too high
PM - WBS / estimation / risk	Breaks down work, estimates, drafts risk lists	What is missing, what must be cut, what is realistic for the real team
All tracks - reports / dashboards	Drafts summaries and KPI views from raw data	Whether the numbers are sourced, honest, and decision-worthy
All tracks - delegation map	Suggests levels and safe zones	The final autonomy level and hazard handling

The pattern is constant: the AI coding agent handles the **boxable** part of the work. Humans keep the **judgment**.

5. Governance overlay — how to keep orchestration safe

The more powerful the orchestrator becomes, the tighter the rails must be.

- **Leash A by default:** the AI coding agent may work up to a draft or bounded execution result, but not self-release.
- **Leash A+ for hazard work:** if a task touches permissions, secrets, release, schema, or sensitive data, it must stop for explicit human approval.
- **Fail-closed gate:** tests, verification, traceability, and security checks must pass before output becomes usable.
- **Hard-stop on contradiction:** if the specification is unclear or conflicting, the AI coding agent must stop and ask.
- **Human-in-the-loop:** important artifacts require a real person who has actually understood the output.

Golden rule:

The more steps AI can run autonomously, the tighter the gate must become.

6. Telemetry and Dev Book — log while you drive

Every serious orchestrator session should produce evidence.

Dev Book

Write down:

- where AI was wrong,
- what you changed,
- why you chose that delegation level,
- which gate blocked the issue,
- whether a hard-stop happened.

This is **more valuable than pretty plans**, because it proves understanding.

Telemetry

Track at least:

- real human hours,
- number of AI override / edit moments,
- estimated token usage and later reconciliation,
- rework.

A learner with 30 real AI corrections is not the same as a learner with zero corrections.
The second case is often hidden rubber-stamping.

7. First-day quickstart — ten steps

1. Open the project or choose one project brief (PB-01 to PB-06).
2. Read or create CLAUDE.md .
3. State the **goal**, not a list of micro-actions.
4. Turn on **Plan mode** for any non-trivial step.
5. Review and edit the plan before execution.
6. Let the AI coding agent execute the step.
7. For any hazard action, force an **A+ approval stop**.
8. Verify the result yourself: test, click, inspect, or reason through it.
9. Record at least one AI mistake and your correction in the Dev Book.
10. Log telemetry and loop forward.

If a learner has completed this cycle once on real work, they have already started operating as an AI orchestrator rather than a chatbot user.

8. Prompt patterns for orchestration

- **End-to-end step delegation**

"Goal: [X]. First present a step-by-step plan and wait for my approval. Once approved, execute directly on the project files. If any assumption is unclear, surface it instead of inventing it."

- **Plan-first control**

"Do not execute yet. Show me the plan, the risks, and the places where I must decide."

- **Boundary setting / leash**

"You may update code and run tests. You may not commit, push, change schema, or touch secrets without asking me."

- **Forced hard-stop**

"If the specification is contradictory or underspecified, stop and list questions. Do not guess."

- **Verification gate**

"After changes, run verification and report the result. If the gate is red, keep fixing until it is green."

- **Parallel work request**

"This task is large. Split independent parts into parallel subagents, then reconcile the outputs for me."

- **Self-review pass**

"Now switch roles and review your own output like a strict reviewer. Show defects, omissions, and unsafe assumptions."

9. Common traps and how to avoid them

Trap	Consequence	Prevention
Copying AI output without reading it	Hidden defects and fake progress	Always verify and record at least one human correction
Letting it run risky work without gates	Security, data, or release accidents	Use A+ and fail-closed gates
Weak prompt with no context	Guessing, drift, shallow output	Good <code>CLAUDE.md</code> + clear goal + plan-first mode
Trusting invented numbers	Bad reports and false estimates	Require explicit source grounding
Micromanaging each command	Wastes the point of orchestration	Delegate the outcome, not the keystrokes
Not logging Dev Book / telemetry	No evidence of understanding	Log during the loop, not at the end

Hard boundary of the bootcamp:

submitting AI-generated work without visible human review and correction is **rubber-stamping** and counts as failure, even if the output looks polished.

10. Pocket checklist

Before delegation

- ☐ Did I load the right context?
- ☐ Did I state a goal, not a keystroke list?
- ☐ Did I request a plan first?
- ☐ Did I define the A / A+ boundary?

While it is running

- ☐ Does hazard work stop for my decision?
- ☐ Did contradictions trigger a hard-stop instead of silent guessing?

Before calling it done

- ☐ Did I verify the result myself?
- ☐ Did I catch and correct at least one AI mistake?
- ☐ Did I log Dev Book and telemetry immediately?

If these three blocks stay true, the learner is using the AI coding agent as a **controlled super orchestrator**, not as a glorified autocomplete engine.

This guide is the entry layer. It works together with the Program (Section 5 Mindset, Section 8 Capstone), the four workbooks (track-specific exercises), and the Operating Model (L0-L5, Leash A/A+, 3 governance mechanisms). For terminology: see [docs/GLOSSARY.md](#).

Changelog

Version	Date	Notes
v1.0	2026-07-02	Initial release
v2.0	2026-08-24	Tool-agnostic rewrite (Claude Code -> AI coding agent); added glossary pointer; added "Applies to" header; updated controls table for tool equivalence; aligned with Operating Model v2.0